

# ATmega32A Module

## Timers (Normal, CTC and Fast PWM)

Ali Sahafi

### 1 Theory

A hardware timer is a counter. It counts clock ticks automatically, and your program does not need to do anything while it counts. The ATmega32A has three timers:

| Timer  | Size   | Counts  | Prescalers                   | OC pin(s)        |
|--------|--------|---------|------------------------------|------------------|
| Timer0 | 8-bit  | 0–255   | 1, 8, 64, 256, 1024          | PB3 (LED D3)     |
| Timer1 | 16-bit | 0–65535 | 1, 8, 64, 256, 1024          | PD5 (A), PD4 (B) |
| Timer2 | 8-bit  | 0–255   | 1, 8, 32, 64, 128, 256, 1024 | PD7              |

**Prescaler.** The prescaler  $N$  divides the CPU clock before it goes to the counter. So one count takes  $t_{\text{count}} = N/F_{\text{CPU}}$ . Example: with 8 MHz and  $N = 1024$ , one count takes  $128 \mu\text{s}$ .

**The three modes.**

- **Normal** (TIMER\_NORMAL): The timer counts up to the maximum (255, or 65535 for Timer1). Then it goes back to 0. This is called an *overflow*, and the timer sets the overflow flag TOV. For a shorter time, you give the counter a start value (a *preload*):

$$t = (256 - \text{preload}) \cdot \frac{N}{F_{\text{CPU}}} \quad (\text{use } 65536 \text{ for Timer1})$$

- **CTC** (TIMER\_CTC, *Clear Timer on Compare*): The timer counts up to the compare value OCR. Then it sets the compare flag OCF and starts again from 0. You do not need a preload:

$$t = (\text{OCR} + 1) \cdot \frac{N}{F_{\text{CPU}}}$$

On every compare match, the timer can also toggle, set or clear its OC pin. This gives a square wave, and the CPU does not need to do anything.

- **Fast PWM** (TIMER\_FAST\_PWM): The timer counts from 0 to 255 again and again. The OC pin changes at 0 and at the compare value. The result is a PWM signal. OCR sets the duty cycle:

$$F_{\text{PWM}} = \frac{F_{\text{CPU}}}{N \cdot 256} \quad \text{Duty cycle (active part)} = \frac{\text{OCR} + 1}{256}$$

### 2 How to use the driver

The driver gives you three objects: Timer0, Timer1 and Timer2. You start a timer with one function call:

```
Timer0.begin(mode, prescaler);           // start the timer
Timer0.begin(mode, prescaler, pinMode);   // start the timer and use the OC pin
```

## The options

| Argument  | Value                                  | Meaning                                                                                           |
|-----------|----------------------------------------|---------------------------------------------------------------------------------------------------|
| mode      | TIMER_NORMAL                           | Count from 0 to the maximum, overflow, start again from 0                                         |
|           | TIMER_CTC                              | Count from 0 to the compare value, start again from 0                                             |
|           | TIMER_FAST_PWM                         | Count from 0 to 255, the OC pin gives a PWM signal                                                |
| prescaler | 1, 8, 64, 256, 1024                    | Write the number you want to divide by. Timer2 can also use 32 and 128.                           |
|           | EXT_FALLING, EXT_RISING                | Only Timer0 and Timer1: count pulses on pin T0 = PB0 or T1 = PB1 (counter mode)                   |
| pinMode   | OC_OFF (default)                       | The OC pin is a normal GPIO pin                                                                   |
|           | <i>with TIMER_NORMAL or TIMER_CTC:</i> |                                                                                                   |
|           | OC_TOGGLE                              | Toggle the OC pin on every compare match                                                          |
|           | OC_CLEAR                               | Set the OC pin LOW on a compare match                                                             |
|           | OC_SET                                 | Set the OC pin HIGH on a compare match                                                            |
|           | <i>with TIMER_FAST_PWM:</i>            |                                                                                                   |
|           | PWM_NON_INVERTING                      | HIGH from 0 to the compare value, then LOW                                                        |
|           | PWM_INVERTING                          | LOW from 0 to the compare value, then HIGH. Use this for the <b>active-low LEDs</b> on the board. |

If `pinMode` is not `OC_OFF`, the driver makes the OC pin an output for you. While the timer controls this pin, `GPIO.write()` has no effect on it.

## Functions

| Function                                                   | What it does                                                                                                                                                                                  |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Timer0.begin(mode, prescaler, pinMode)                     | Set up and start the timer (see the options above)                                                                                                                                            |
| Timer0.setCount(value)                                     | Write a start value to the counter (example: 156 gives 100 counts in Normal mode)                                                                                                             |
| Timer0.getCount()                                          | Read the counter (0–255; Timer1: 0–65535)                                                                                                                                                     |
| Timer0.setCompare(value)                                   | Set the compare value. CTC: the timer counts up to this value. PWM: the level, 0–255                                                                                                          |
| Timer0.setDutyCycle(percent)                               | Only PWM: how many percent (0–100) of the time the pin is <i>active</i> . Active is HIGH for <code>PWM_NON_INVERTING</code> and LOW for <code>PWM_INVERTING</code> . 0 % and 100 % are exact. |
| Timer0.overflowed()                                        | Returns <b>true</b> one time for each overflow. The flag is cleared for you                                                                                                                   |
| Timer0.compareMatched()                                    | Returns <b>true</b> one time for each compare match. The flag is cleared for you                                                                                                              |
| Timer0.stop() / Timer0.start()                             | Stop / continue counting. The settings do not change                                                                                                                                          |
| Timer0.onOverflow(function)                                | <i>Optional:</i> call <b>function</b> on every overflow (interrupt)                                                                                                                           |
| Timer0.onCompareMatch(function)                            | <i>Optional:</i> call <b>function</b> on every compare match (interrupt)                                                                                                                      |
| <i>Only Timer1 (it has two OC pins: A = PD5, B = PD4):</i> |                                                                                                                                                                                               |
| Timer1.begin(mode, N, pinModeA, pinModeB)                  | Channel B has its own <code>pinMode</code>                                                                                                                                                    |
| Timer1.setCompareA/B(value)                                | 16-bit compare values. <code>setCompare()</code> is channel A. In CTC mode the timer counts up to A                                                                                           |
| Timer1.setDutyCycleA/B(percent)                            | PWM duty cycle for each channel. <code>setDutyCycle()</code> is channel A                                                                                                                     |
| Timer1.compareMatchedA/B()                                 | Compare flag for each channel. <code>compareMatched()</code> is channel A                                                                                                                     |
| Timer1.setTop(top)                                         | Length of the PWM period in counts (default 255):<br>$T_{\text{PWM}} = (\text{top} + 1) \cdot N / F_{\text{CPU}}$                                                                             |

**Waiting for a flag (polling).** `overflowed()` and `compareMatched()` return **true** when the event happened after your last call. So `while (!Timer0.overflowed());` waits until the next overflow. You do not need to write 1 to TIFR.

**Interrupts (optional).** `onOverflow()` and `onCompareMatch()` turn on the interrupt and call `sei()` for you. Your function then runs automatically. Keep this function short. If it changes a variable that `main()` also uses, declare the variable as `volatile`. To turn the interrupt off again, pass `nullptr`.

## Driver functions and registers

In the lecture slides we set the registers by hand. Each driver function does one of these steps for you:

| Driver function                                             | Registers (Timer0)                      | Bits                                    |
|-------------------------------------------------------------|-----------------------------------------|-----------------------------------------|
| <code>Timer0.begin(TIMER_NORMAL, 1024)</code>               | <code>TCCR0 = 0x05;</code>              | CS02 CS00                               |
| <code>Timer0.begin(TIMER_CTC, 1024, OC_TOGGLE)</code>       | <code>TCCR0 = 0x1D;</code>              | WGM01 COM00 CS02 CS00                   |
|                                                             | <code>DDRB  = 1&lt;&lt;PB3;</code>      | OC0 pin as output                       |
| <code>Timer0.begin(TIMER_FAST_PWM, 1, PWM_INVERTING)</code> | <code>TCCR0 = 0x79;</code>              | WGM00 WGM01 COM01 COM00 CS00            |
| <code>Timer0.setCount(156)</code>                           | <code>TCNT0 = 156;</code>               |                                         |
| <code>Timer0.setCompare(x)</code>                           | <code>OCR0 = x;</code>                  |                                         |
| <code>Timer0.overflowed()</code>                            | <code>TIFR &amp; (1&lt;&lt;TOV0)</code> | then <code>TIFR = 1&lt;&lt;TOV0;</code> |
| <code>Timer0.compareMatched()</code>                        | <code>TIFR &amp; (1&lt;&lt;OCF0)</code> | then <code>TIFR = 1&lt;&lt;OCF0;</code> |

**Important: F\_CPU and the fuses.** All timing formulas use the *real* clock of the chip. `make` and `make timer` always set the fuses for the external 8 MHz crystal. To use another clock: (1) set the fuses, for example `make int-1mhz`; (2) upload your program without changing the fuses: `make build flash SRC=Task.cpp`; (3) change `#define F_CPU` to the new clock, so that `_delay_ms()` is still correct.

### 3 Example: blinking LED and PWM, both done by the timers

**Board hardware.** All parts are already on the development board:

- **8 LEDs (D0–D7) on PORTB.** They are active-low: LOW = ON. LED **D3** is on PB3 = OC0, the output pin of Timer0.
- **Seven-segment display**, also on PORTB (it uses the same pins as the LEDs).
- **8 DIP switches (S0–S7) on PORTC.** A closed switch reads LOW. Use INPUT\_PULLUP.
- **Push buttons:** S11 on PD2 and S10 on PD6. A pressed button reads LOW.

**Build and upload.** `make timer`

```
/*
 * Timer Example -- one timer per mode, all running at the same time
 * Build & flash: make timer
 *
 * LED D0 (PB0): blinks every 0.5 s -- Timer1, CTC mode, flag polling
 * LED D3 (PB3): half brightness -- Timer0, Fast PWM on its OC0 pin
 *
 * Timer1 CTC period = prescaler * (compare + 1) / F_CPU
 *                   = 256 * (15624 + 1) / 8 MHz = 0.5 s
 */

#define F_CPU 8000000UL
#include "../gpio/gpio.hpp"
#include "timer.hpp"

int main() {
    GPIO.setDirection(PORTB, PB0, OUTPUT); // LED D0 (active-low)

    // Timer0: Fast PWM, prescaler 64 -> 8 MHz / (64 * 256) = 488 Hz, no flicker.
    // PWM_INVERTING because the LEDs are active-low (duty cycle = brightness).
    Timer0.begin(TIMER_FAST_PWM, 64, PWM_INVERTING); // PB3 becomes an output
    Timer0.setDutyCycle(50);

    // Timer1: CTC, restarts every 15625 counts = 0.5 s
    Timer1.begin(TIMER_CTC, 256);
    Timer1.setCompare(15624);

    while (true) {
        if (Timer1.compareMatched()) { // true once every 0.5 s
            GPIO.toggle(PORTB, PB0);
        }
        // The CPU is free here -- the PWM on D3 needs no code at all.
    }
}
```

## 4 Laboratory Tasks

For each task, make a new `.cpp` file in the main project folder (for example `Task1.cpp`). Start the file with `#define F_CPU`. Then include the drivers you need: `drivers/gpio/gpio.hpp`, `drivers/timer/timer.hpp`, `drivers/common/sevenseg.hpp`. Upload it with `make SRC=Task1.cpp`.

### Task 0: Fuse Bits and F\_CPU (10 min)

Show the numbers 0 to 9 on the seven-segment display. Change the number every second. Use `SevenSeg(digit)` and `_delay_ms(1000)`.

- **Task 0.1 (Internal 1 MHz):** Run `make int-1mhz`. Write `#define F_CPU 1000000UL` and upload with `make build flash SRC=Task0.cpp`. Check the time with a stopwatch.
- **Task 0.2 (External 8 MHz crystal):** Write `#define F_CPU 8000000UL` and upload with `make SRC=Task0.cpp`. This command also selects the crystal.
- **Task 0.3 (Wrong F\_CPU):** Keep `F_CPU 8000000UL`, but change the chip back to 1 MHz with `make int-1mhz`. How long is one “second” now? Explain why.

### Task 1: Timer Normal Mode — Counter on the LEDs (30 min)

Count in binary on the 8 LEDs. Add 1 every **100 ms**. Use `Timer0` in `TIMER_NORMAL` mode for the timing (external 8 MHz crystal). The LEDs are active-low, so write the inverted value: `GPIO.write(PORTB, ~count)`.

- **Task 1.1 (Calculate first):** With prescaler 1024, how long is one count? What is the *longest* time between two `Timer0` overflows? Why is one overflow not enough for 100 ms?
- **Task 1.2 (10 × 10 ms):** Calculate the preload value for 10 ms. In the main loop, do these two steps 10 times:
  1. write the preload with `Timer0.setCount()`;
  2. wait with `while (!Timer0.overflowed());`.

After that, add 1 to the counter.

- **Task 1.3 (16-bit Timer1):** Do the same with `Timer1`. `Timer1` can make 100 ms with *one* overflow. With which prescalers is the preload value a whole number? Compare your code with Task 1.2.

### Task 2: Timer CTC Mode — Blinking in Hardware (20 min)

The timer blinks LED **D3** (`PB3 = 0C0`) without help from the CPU. The DIP switches on `PORTC` set the blink speed.

- **Task 2.1 (Blink with the timer):** Start `Timer0` in `TIMER_CTC` mode with prescaler 1024 and `OC_TOGGLE`. The main loop only does one thing: it reads the switches (`GPIO.read(PINC)`) and writes the value with `Timer0.setCompare()`. You do not need to check any flag. At 8 MHz, calculate the slowest possible blink frequency. What do you see on D3? Explain why.
- **Task 2.2 (Make it visible):** Change the chip to the internal 1 MHz clock (see Task 0.1). What is the shortest and the longest time between two toggles now? Test it with the switches.
- **Task 2.3 (Two things at the same time):** Keep the blinking LED. In the same loop, also show the switch value on the other LEDs: `GPIO.write(PORTB, GPIO.read(PINC))`. The loop also writes to `PB3`. Why does D3 still blink?

### Task 3: Timer Fast PWM — LED Brightness (25 min)

Change the brightness of LED **D3** (OC0) with PWM from `Timer0`.

- **Task 3.1 (Switches → brightness):** Start `Timer0` in `TIMER_FAST_PWM` mode with `PWM_INVERTING`, because the LED is active-low. Choose the prescaler so that  $F_{\text{PWM}} \approx 4 \text{ kHz}$  at 8 MHz. In the loop, write `Timer0.setCompare(GPIO.read(PINC))`.
- **Task 3.2 (Buttons, in percent):** Use the push buttons instead of the switches. S11 (PD2) makes the LED 10 % brighter. S10 (PD6) makes it 10 % darker. Keep the value between 0 and 100 %. Use `Timer0.setDutyCycle(percent)`. One press must change the value only one time, so wait until the button is released.
- **Task 3.3 (Flicker):** Calculate  $F_{\text{PWM}}$  for the prescalers 64, 256 and 1024. Try all three. With which prescaler do you start to see the LED flicker?
- **Task 3.4 (Breathing LED with two timers):** Make D3 slowly brighter from 0 to 100 %, then slowly darker again. Change the value by 1 % every 10 ms. Do not use `_delay_ms()`. Instead, use `Timer1` in `TIMER_CTC` mode to measure the 10 ms (`Timer1.compareMatched()`). Calculate the prescaler and the compare value for `Timer1`.